Announcements 📣

- Midterm Exam
  - Date: April 2nd
  - Time: 7:00-9:00 (pm)
  - Location: B22 (Rooms 119, 125,127, 130, 134)
- Command to get process access in the lab PCs

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Web Engineering & Development (SWE 363)

Interactive Front-End Development

In today's lecture:

- Event handling
- State

Reference:

- Zybook: 5.8-5.9

5.8 Event handling

What is Event Handling?

Event handling allows your React components to respond to user interactions

Examples of Events:

- User clicks a button
- User types in an input field
- User submits a form
- User hovers over an element

Why do we need it?

- Makes web pages **interactive**
- Responds to user actions
- Creates dynamic user experiences

Event Handling in React vs HTML

HTML Event Handling:

```
<button onclick="handleClick()">Click Me</button>
```

React Event Handling:

```
<button onClick={handleClick}>Click Me</button>
```

Key Differences:

- **camelCase** naming (`onClick` not `onclick`)
- **Function reference** not string (`{handleClick}` not `"handleClick()"`)
- **Curly braces** around the function name

Common React Events

Event	When it triggers	Example
`onClick`	Element is clicked	Button clicks
`onChange`	Input value changes	Typing in text field
`onSubmit`	Form is submitted	Form submission
`onMouseEnter`	Mouse enters element	Hover effects
`onMouseLeave`	Mouse leaves element	Hover effects

Writing Event Handler Functions

```
function MyComponent() {
  function handleClick() {
    console.log('Button was clicked!');
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

Event Handler with Parameters

```
function TaskList() {
  const tasks = ['Task 1', 'Task 2', 'Task 3'];

  function handleDelete(taskIndex) {
    console.log(`Deleting task at index: ${taskIndex}`);
  }
  return (
    <ul>
      {tasks.map((task, index) => (
        <li key={index}>
          {task}
          <button onClick={() => handleDelete(index)}> Delete </button>
        </li>
      ))}
    </ul>
  );
}
```

Input Event Handling

Handling text input changes:

```
function TextInput() {
  function handleChange(event) {
    console.log('Input value:', event.target.value);
  }

  return (
    <input
      type="text"
      onChange={handleChange}
      placeholder="Type something..."
    />
  );
}
```

Form Event Handling

Handling form submission:

```
function ContactForm() {
  function handleSubmit(event) {
    event.preventDefault(); // Prevents page refresh
    console.log('Form submitted!');
  }
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" placeholder="Your name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

`event.preventDefault()` stops the default form behavior

5.9 State

What is State?

State is data that can change over time in a React component

Key Characteristics:

- **Mutable** - can be updated
- **Component-specific** - belongs to one component
- **Triggers re-renders** - when state changes, component re-renders
- **Persistent** - survives re-renders

Examples of State:

- Counter value
- User input text
- List of items
- Toggle visibility

State vs Props

State	Props
Internal to component	Passed from parent
Can be changed	Read-only
Managed with `useState`	Passed as attributes
Triggers re-render	Triggers re-render

State vs Props

Example:

```
function Counter({ initialValue }) { // props
  const [count, setCount] = useState(initialValue); // state
  return <div>{count}</div>;
}
```

Using useState Hook

Import useState:

```
import { useState } from 'react';
```

Basic Syntax:

```
function MyComponent() {
  const [stateVariable, setStateFunction] = useState(initialValue);

  return <div>{stateVariable}</div>;
}
```

Using useState Hook

Example:

```
function Counter() {
  const [count, setCount] = useState(0);
  return <div>Count: {count}</div>;
}
```

What each part means:

- `count` - **state variable** (current value)
- `setCount` - **setter function** (updates the state)
- `useState(0)` - **initial value** (starts at 0)
- `[count, setCount]` - **array destructuring**

Using useState Hook

Example:

```
function Counter() {
  const [count, setCount] = useState(0);
  return <div>Count: {count}</div>;
}
```

Naming Convention:

- State variable: `count` , `name` , `items`
- Setter function: `setCount` , `setName` , `setItems`

Updating State

Correct way to update state:

```
function Counter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount(count + 1); // Correct
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+1</button>
    </div>
  );
}
```

Updating State

Wrong way:

```
function increment() {
  count = count + 1; // Wrong! Don't modify state directly
}
```

State with Different Data Types

String State:

```
const [name, setName] = useState('');
setName('John');
```

Number State:

```
const [age, setAge] = useState(0);
setAge(25);
```

State with Different Data Types

Boolean State:

```
const [isVisible, setIsVisible] = useState(false);
setIsVisible(true);
```

Array State:

```
const [items, setItems] = useState([]);
setItems(['item1', 'item2']);
```

State with Objects

Object State:

```
const [user, setUser] = useState({ name: '', email: '' });

// Update entire object
setUser({ name: 'John', email: 'john@example.com' });

// Update specific property
setUser({ ...user, name: 'Jane' });
```

State with Objects

Array of Objects:

```
const [tasks, setTasks] = useState([
  { id: 1, text: 'Learn React', completed: false },
  { id: 2, text: 'Build app', completed: false }
]);
```

Adding Items to Array State

Adding to array:

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [inputValue, setInputValue] = useState('');

  function addTodo() {
    const newTodo = { id: Date.now(), text: inputValue, completed: false };
    setTodos([...todos, newTodo]);
    setInputValue(''); // Clear input
  }

  return (<div>
      <input value={inputValue} onChange={(e) => setInputValue(e.target.value)} />
      <button onClick={addTodo}>Add Todo</button>
    </div>);
}
```

Removing Items from Array State

Remove by ID:

```
function removeTodo(id) {
  setTodos(todos.filter(todo => todo.id !== id));
}
```

Remove all items:

```
function clearAll() {
  setTodos([]);
}
```

Updating Items in Array State

Update specific item:

```
function toggleComplete(id) {
  setTodos(todos.map(todo =>
    todo.id === id
      ? { ...todo, completed: !todo.completed }
      : todo
  ));
}
```

State Best Practices

1. Don't mutate state directly:

```
todos.push(newTodo); // Wrong
setTodos(todos); // Wrong

setTodos([...todos, newTodo]); // Correct
```

2. Keep state minimal:

```
// Too many states
const [firstName, setFirstName] = useState('');
const [lastName, setLastName] = useState('');

// Better state
const [name, setName] = useState({ first: '', last: '' });
```

Common State Patterns

Toggle Pattern:

```
const [isVisible, setIsVisible] = useState(false);

function toggle() {
  setIsVisible(!isVisible);
}
```

Counter Pattern:

```
const [count, setCount] = useState(0);

function increment() { setCount(count + 1); }
function decrement() { setCount(count - 1); }
function reset() { setCount(0); }
```

Common State Patterns

Form Pattern:

```
const [formData, setFormData] = useState({ name: '', email: '' });

function handleChange(field, value) {
  setFormData({ ...formData, [field]: value });
}
```

30m

Demo

5.4 More React

Next Class

- More on React